

Volume 3

Git, GitHub and Networking for DevOps

Beginner to Professional | Hands-on Labs | Enterprise Workflows | Interview Mastery

Target learner	Absolute beginner progressing to production-ready DevOps engineer
Core outcomes	Professional Git, GitHub collaboration, networking fundamentals, troubleshooting and security
Practice model	Read -> reproduce -> explain -> troubleshoot -> document -> mark completed
Completion evidence	Labs, capstones, practical examination and interview scenarios

This volume is designed for continuous study. Mark each unit only after completing its commands, lab and review questions.

How to Use This Book

Study one unit at a time. Do not only read commands: type them, inspect the output, deliberately create a failure, recover from it, and write a short explanation in your own words. A unit is complete only when you can repeat its lab without copying every step.

Stage	Required action	Evidence
Learn	Read concepts and annotate key terms	Personal notes
Practice	Run commands in a disposable repository or lab VM	Terminal history or screenshots
Explain	Describe the workflow without reading	Written summary
Troubleshoot	Create and resolve at least one failure	Incident note
Validate	Answer knowledge-check questions	Self-score of 80% or more
Complete	Mark unit completed and record date	Tracker entry

Laboratory Requirements

- A Windows, Linux or macOS computer with Git installed.
- A GitHub account using multi-factor authentication.
- A Linux VM or WSL environment for networking labs.
- Administrator rights only where a lab explicitly needs them.
- A separate practice repository. Never experiment in a production repository.

Course Map

Part	Focus	Units
Part I	Git foundations and local version control	Units 1-10
Part II	Collaboration and advanced Git	Units 11-18
Part III	GitHub and enterprise governance	Units 19-25
Part IV	Networking foundations	Units 26-35
Part V	DevOps networking, security and troubleshooting	Units 36-43
Part VI	Capstones, interview mastery and final examination	Units 44-48

Unit 1: Version Control and the DevOps Software Lifecycle

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Explain why version control exists.
- Compare local, centralized and distributed version control.
- Place Git inside a modern DevOps delivery pipeline.

Why this matters in DevOps

DevOps depends on traceability. Infrastructure code, application code, configuration, deployment manifests and operational documentation must all have controlled history.

The problem before version control

Without version control, teams overwrite files, lose working versions, cannot prove who changed what, and cannot reliably reproduce releases. File names such as final_v2_really_final are not a governance system.

Three version-control models

- Local version control stores history on one machine.
- Centralized version control uses one central server; clients check out working copies.
- Distributed version control gives each clone a complete repository history, improving resilience and offline work.

Git in the delivery chain

A typical flow is: issue -> branch -> commits -> pull request -> automated checks -> review -> merge -> build artifact -> deployment -> monitoring -> rollback if needed.

How companies use this

Companies connect commits and pull requests to tickets, CI results, security scans, releases and production incidents. This produces an auditable chain from requirement to running change.

Common mistakes and prevention

- Treating Git as only a backup tool.
- Committing generated artifacts or secrets without policy.
- Making production changes outside version control.

Hands-on lab

1. Install Git and verify the version.
2. Create a folder named volume3-labs.
3. Write a one-page note describing how a code change moves from developer to production.
4. Identify five assets besides application source code that belong in version control.

Knowledge check

5. What is the difference between backup and version control?
6. Why is distributed history useful?
7. Which DevOps artifacts should be versioned?

Completion evidence: _____

Unit 2: Installing and Configuring Git

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Install Git correctly.
- Configure identity and default behavior.
- Distinguish system, global and local configuration.

Why this matters in DevOps

Correct configuration prevents inconsistent branch names, broken line endings and commits with incorrect identity.

Configuration scopes

System settings affect all users. Global settings affect one user. Local settings affect one repository and override broader scopes.

Identity and behavior

Git records author and committer metadata. Teams also standardize default branch names, pull strategy, editor and line-ending behavior.

Command laboratory

Inspect installation

```
git --version
git config --list --show-origin
```

The origin column shows which configuration file supplied each value.

Set baseline configuration

```
git config --global user.name "Sunny"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --global pull.rebase false
```

Use your real professional identity. Do not use placeholder email in actual work.

Inspect scopes

```
git config --system --list
git config --global --list
git config --local --list
```

The local command must be run inside a repository.

Common mistakes and prevention

- Using different identities across machines unintentionally.
- Ignoring line-ending differences between Windows and Linux.
- Changing system configuration when only repository configuration is needed.

Hands-on lab

8. Configure your real name and email.
9. Create two repositories and set a different local email in one.
10. Use --show-origin to prove which configuration wins.

Knowledge check

11. Which scope has the highest priority?
12. Why might a company require a local repository email?
13. What does `init.defaultBranch` control?

Completion evidence: _____

Unit 3: Repositories, Working Tree, Index and Commit History

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Explain the three main local states.
- Create a repository and inspect status.
- Move changes through working tree, index and repository.

Why this matters in DevOps

Most Git confusion comes from not understanding where a change currently lives.

The three-state model

The working tree contains editable files. The index, also called staging area, is the proposed next snapshot. The repository stores committed history.

Snapshot thinking

Git commits are snapshots connected by parent relationships, not merely patches. Git can calculate differences between snapshots when needed.

Command laboratory

Create repository

```
mkdir git-state-lab
cd git-state-lab
git init
echo '# Git State Lab' > README.md
git status
```

The file is untracked until added.

Stage and commit

```
git add README.md
git status
git commit -m "docs: add initial README"
git log --oneline --decorate
```

The commit records the staged snapshot.

Compare states

```
echo 'Second line' >> README.md
git diff
git add README.md
git diff --staged
```

git diff compares working tree to index; --staged compares index to HEAD.

How companies use this

Code reviews evaluate the staged and committed changes that become part of a pull request. CI systems build from commits, not from a developer's uncommitted working tree.

Common mistakes and prevention

- Assuming git add means upload.
- Committing all files with git add . without reviewing them.
- Confusing untracked, modified and staged states.

Hands-on lab

14. Create two files and commit only one.
15. Modify both files, stage selected lines using git add -p, and inspect both diffs.
16. Write a diagram of working tree -> index -> repository.

Knowledge check

17. What exactly does git add do?
18. Which command shows staged changes?
19. Can CI see your uncommitted working-tree changes?

Completion evidence: _____

Unit 4: Commit Design, Messages and Atomic Changes

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Create atomic commits.
- Write professional commit messages.
- Use commit metadata responsibly.

Why this matters in DevOps

Small, coherent commits make reviews, reverts, debugging and release notes safer.

Atomic commit principle

One commit should represent one logical change. A change may touch several files, but they should serve the same purpose.

Message structure

A useful message has a concise imperative subject, optional explanation of why, and references to issues when appropriate.

Conventional style

Many teams use prefixes such as feat, fix, docs, test, refactor, build, ci and chore. The exact convention is less important than consistency.

Command laboratory

Review before commit

```
git status
git diff
git diff --staged
```

Review content before creating history.

Commit with body

```
git commit -m "fix: prevent empty deployment target" -m "Validate the environment before invoking the
release command."
```

The body explains motivation and important consequences.

Amend safely

```
git commit --amend
```

Amend unpublished local commits. Avoid rewriting shared history without coordination.

How companies use this

Organizations may validate commit formats, require signed commits, link commits to tickets, and generate changelogs from structured messages.

Common mistakes and prevention

- Messages such as update or changes.
- Mixing formatting, feature work and unrelated fixes.
- Amending a shared commit and force-pushing without coordination.

Hands-on lab

20. Create three atomic commits in a sample project.
21. Write a subject and explanatory body for each.
22. Use `git show --stat` to review the history.

Knowledge check

23. What makes a commit atomic?
24. Why should a message explain why?
25. When is amend dangerous?

Completion evidence: _____

Unit 5: Git Objects and Internal Architecture

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Identify blob, tree, commit and tag objects.
- Understand content-addressable storage.
- Inspect repository objects safely.

Why this matters in DevOps

Git internals explain why identical content is reused, why commits are immutable and how history is connected.

Object types

- A blob stores file content, not its filename.
- A tree maps names to blobs and subtrees.
- A commit references a root tree, parent commit or commits, author, committer and message.
- An annotated tag object can point to another object and carry metadata and a signature.

Content addressing

Objects are named by cryptographic hashes derived from their type, length and content. Changing the content produces a different object identifier.

References

Branches and tags are human-friendly references to object IDs. HEAD usually points symbolically to the current branch.

Command laboratory

Inspect object IDs

```
git rev-parse HEAD
git cat-file -t HEAD
git cat-file -p HEAD
```

cat-file reveals the type and content of an object.

Inspect tree

```
git ls-tree HEAD
git rev-parse HEAD^{tree}
```

The tree connects path names to object IDs.

Hash a file

```
git hash-object README.md
```

This computes the blob ID without changing the file.

Common mistakes and prevention

- Editing files inside .git manually.
- Assuming a branch stores copies of every file.
- Believing hashes are random identifiers.

Hands-on lab

26. Inspect the current commit, tree and one blob.

27. Change one file and compare object IDs.

28. Draw the graph commit -> tree -> blob.

Knowledge check

29. Where is a filename stored?

30. Why does a commit change when its tree changes?

31. What does HEAD usually reference?

Completion evidence: _____

Unit 6: Branches, HEAD and Detached HEAD

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Create and switch branches.
- Explain branch pointers.
- Recover safely from detached HEAD.

Why this matters in DevOps

Branches isolate streams of work and allow multiple changes to progress without overwriting one another.

Branch model

A branch is a movable reference to a commit. Creating a commit advances the currently checked-out branch.

HEAD

HEAD identifies the current checkout. Normally it points to a branch. In detached HEAD state it points directly to a commit.

Branch naming

Professional teams use readable names tied to work, for example feature/login-rate-limit or fix/payment-timeout.

Command laboratory

Create branches

```
git switch -c feature/readme-details
echo 'Details' >> README.md
git add README.md
git commit -m "docs: expand README"
git branch -vv
```

switch -c creates and checks out a branch.

Detached HEAD

```
git switch --detach HEAD~1
git status
git switch main
```

Detached checkout is useful for inspection, but new commits need a branch if they must be retained.

Recover a detached commit

```
git switch -c recovery/important-work
```

Create a branch before leaving the detached commit.

How companies use this

Teams map branches to pull requests, automated preview environments and policy checks. Long-lived branches require careful synchronization and increase integration risk.

Common mistakes and prevention

- Creating work in detached HEAD and losing the reference.

- Using vague branch names.
- Keeping feature branches open for weeks without updating them.

Hands-on lab

32. Create two feature branches from main.
33. Make different commits on each.
34. Enter detached HEAD, create a commit, and preserve it with a new branch.

Knowledge check

35. Is a branch a directory?
36. What moves when you commit?
37. How do you save a detached commit?

Completion evidence: _____

Unit 7: Merging and Merge Conflicts

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Perform fast-forward and three-way merges.
- Resolve conflicts systematically.
- Validate a merge before publishing it.

Why this matters in DevOps

Integration is where independent work becomes one tested history.

Fast-forward merge

When the target branch has not diverged, its pointer can move directly to the source tip.

Three-way merge

When both branches have new work, Git compares their tips and common ancestor and creates a merge commit.

Conflict resolution

A conflict means Git needs a human decision. Resolve meaning, not only markers. Then test the combined behavior.

Command laboratory

Merge a feature

```
git switch main
git merge feature/readme-details
```

Inspect the output to identify fast-forward or merge commit behavior.

Conflict workflow

```
git status
git diff
# edit conflicted files
git add <resolved-file>
git commit
```

git status is the primary guide during conflict resolution.

Abort merge

```
git merge --abort
```

Abort when the integration must be restarted from a clean state.

How companies use this

Protected branches usually require pull requests, automated checks and approvals before the server performs or permits a merge.

Common mistakes and prevention

- Removing conflict markers without understanding both changes.
- Merging with failing tests.
- Resolving generated lockfiles casually without regeneration.

Hands-on lab

38. Create two branches that edit the same line.
39. Merge and resolve the conflict.
40. Run a validation command and record the final graph using `git log --graph --oneline --all`.

Knowledge check

41. What three commits participate in a three-way merge?
42. What does `git merge --abort` do?
43. Why must a resolved merge be tested?

Completion evidence: _____

Unit 8: Rebase and History Rewriting

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Explain rebase accurately.
- Rebase a private feature branch.
- Use force-with-lease instead of unsafe force.

Why this matters in DevOps

Rebase can simplify local history, but it replaces commits and therefore must be governed carefully.

What rebase does

Rebase identifies commits unique to the current branch, resets the branch onto a new base, and replays equivalent changes as new commits.

Private versus shared history

Rewriting unpublished work is usually safe. Rewriting commits other people already use causes divergence and recovery work.

Interactive rebase

Interactive mode can reorder, squash, reword, edit or drop commits before publication.

Command laboratory

Update a feature branch

```
git switch feature/readme-details
git fetch origin
git rebase origin/main
```

Resolve conflicts, add files and continue with `git rebase --continue`.

Interactive cleanup

```
git rebase -i HEAD~3
```

Use only on commits that are safe to rewrite.

Safer force push

```
git push --force-with-lease
```

This refuses to overwrite remote work you have not seen.

How companies use this

Some teams prefer rebase-and-merge for a linear main branch; others preserve merge commits for integration context. The policy should be explicit.

Common mistakes and prevention

- Rebasing a shared branch.
- Using `git push --force` on protected or collaborative branches.
- Squashing useful troubleshooting context into one enormous commit.

Hands-on lab

44. Create three commits on a feature branch.
45. Use interactive rebase to reword and squash them.

46. Compare old and new commit IDs using `reflog`.

Knowledge check

47. Why do commit IDs change during rebase?

48. When should rebase be avoided?

49. Why is `--force-with-lease` safer?

Completion evidence: _____

Unit 9: Undoing Changes: Restore, Reset, Revert and Reflog

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Choose the correct undo tool.
- Recover deleted or rewritten history.
- Distinguish local history editing from shared-history reversal.

Why this matters in DevOps

Production engineers must recover safely without destroying teammates' work.

Restore

git restore changes working-tree or staged content depending on options.

Reset

git reset moves a branch and optionally adjusts index and working tree. Soft, mixed and hard modes have different effects.

Revert

git revert creates a new commit that applies the inverse of an earlier commit. It is normally safest for shared history.

Reflog

Reflog records recent local reference movements and is a powerful recovery mechanism.

Command laboratory

Discard unstaged change

```
git restore README.md
```

This is destructive for uncommitted content.

Unstage while keeping work

```
git restore --staged README.md
```

The working-tree modification remains.

Revert published commit

```
git revert <commit-id>
```

A new commit records the reversal.

Recover with reflog

```
git reflog
```

```
git branch recovery <object-id>
```

Create a branch at the desired previous state before experimenting further.

How companies use this

Incident responders often use revert to restore production quickly, then create a separate forward fix after analysis.

Common mistakes and prevention

- Using reset --hard without checking uncommitted work.

- Resetting a published branch and force-pushing.
- Assuming reflog exists on every remote or forever.

Hands-on lab

50. Make and discard an unstaged edit.
51. Create a bad commit and revert it.
52. Reset a private branch, then recover the prior tip using reflog.

Knowledge check

53. Which command is safest for undoing a shared commit?
54. What does `reset --soft` preserve?
55. Is reflog shared with GitHub?

Completion evidence: _____

Unit 10: Tags, Releases, Archives and Versioning

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Create lightweight and annotated tags.
- Explain release immutability.
- Use semantic versioning appropriately.

Why this matters in DevOps

Deployments need durable identifiers that connect source, artifacts, change records and rollback targets.

Tags

A lightweight tag is a direct reference. An annotated tag is a first-class object with tagger, date, message and optional signature.

Semantic versioning

A common form is MAJOR.MINOR.PATCH: incompatible change, backward-compatible feature, backward-compatible fix. Real policies may include pre-release identifiers.

Release integrity

A release should point to an exact commit, and built artifacts should be traceable to that commit and preferably immutable.

Command laboratory

Create annotated tag

```
git tag -a v1.0.0 -m "Release version 1.0.0"
git show v1.0.0
```

Annotated tags are preferred for formal releases.

Push tags

```
git push origin v1.0.0
git push origin --tags
```

Push only intended tags in controlled workflows.

Create source archive

```
git archive --format=tar.gz --output=release-v1.0.0.tar.gz v1.0.0
```

Archive exports tracked files from the tagged tree.

How companies use this

Release pipelines can trigger from protected tags, build signed artifacts, publish checksums and update deployment records.

Common mistakes and prevention

- Moving an already published release tag.
- Tagging an untested commit.
- Confusing a Git tag with a GitHub Release page.

Hands-on lab

56. Create v0.1.0 and v0.1.1 annotated tags.

57. Generate an archive from one tag.

58. Write a release note with changes, risks and rollback reference.

Knowledge check

59. What extra data does an annotated tag contain?

60. Why should release tags be immutable?

61. What does PATCH mean in semantic versioning?

Completion evidence: _____

Unit 11: Remote Repositories, Clone, Fetch, Pull and Push

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Connect local and remote repositories.
- Distinguish fetch from pull.
- Understand remote-tracking branches.

Why this matters in DevOps

Distributed collaboration requires precise understanding of which references are local and which represent last-known remote state.

Remote model

A remote is a named set of repository URLs. origin is only a convention. origin/main is a local remote-tracking reference updated by fetch.

Fetch and pull

fetch downloads objects and updates remote-tracking references. pull performs fetch followed by integration using merge or rebase.

Push

push proposes updates to remote references. The server can reject non-fast-forward updates or policy violations.

Command laboratory

Inspect remotes

```
git remote -v
git remote show origin
```

Review fetch and push URLs and branch relationships.

Synchronize safely

```
git fetch --prune origin
git log --oneline --left-right main...origin/main
```

Inspect divergence before integrating.

Push branch

```
git push -u origin feature/network-docs
```

-u records the upstream relationship.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Hands-on lab

62. Clone a practice repository.
63. Create a branch, push it and inspect upstream configuration.
64. Delete a remote branch and prune the local remote-tracking reference.

Knowledge check

65. What does fetch change?

66. What is origin/main?

67. Why can a push be rejected?

Completion evidence: _____

Unit 12: Stash, Clean and Worktree

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Temporarily store unfinished work.
- Remove untracked content safely.
- Use multiple worktrees for parallel tasks.

Why this matters in DevOps

Operational work often requires switching context without mixing partial changes.

Stash

Stash stores selected working-tree and index changes in a stack. It is useful for short-lived context switching, not long-term collaboration.

Clean

git clean removes untracked files. Always preview first.

Worktree

Additional worktrees allow multiple branches to be checked out simultaneously from one repository.

Command laboratory

Stash workflow

```
git stash push -u -m "WIP: network notes"
git stash list
git stash show -p stash@{0}
git stash pop
```

-u includes untracked files.

Clean preview

```
git clean -nd
git clean -fd
```

Run destructive clean only after inspecting the preview.

Create worktree

```
git worktree add ../hotfix-worktree -b hotfix/urgent main
```

A separate directory checks out the new branch.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Hands-on lab

68. Create tracked and untracked changes and stash both.
69. Recover them with apply and then drop the stash.
70. Create a worktree for a hotfix branch.

Knowledge check

71. Why is clean -n important?

72. What is the difference between stash apply and pop?

73. When is worktree better than stash?

Completion evidence: _____

Unit 13: Cherry-pick, Bisect and Blame

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Apply a selected commit.
- Locate a regression efficiently.
- Use blame as context, not accusation.

Why this matters in DevOps

These tools support release branching, production fixes and root-cause analysis.

Cherry-pick

Cherry-pick applies the change represented by a commit onto the current branch, producing a new commit.

Bisect

Bisect performs binary search between known good and bad commits to locate the first bad commit.

Blame

Blame maps lines to commits. The commit discussion and surrounding history are more useful than the author's name alone.

Command laboratory

Cherry-pick

```
git switch release/1.x
git cherry-pick <fix-commit>
```

Resolve conflicts and validate the target branch.

Manual bisect

```
git bisect start
git bisect bad
git bisect good <known-good>
# test each checkout, then mark good or bad
git bisect reset
```

Automated tests can be supplied using git bisect run.

Inspect line history

```
git blame -L 10,25 path/to/file
git show <commit-id>
```

Follow the commit to understand intent.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Hands-on lab

74. Create a deliberate regression across several commits and find it with bisect.
75. Cherry-pick a fix into another branch.

76. Use blame and show to write a neutral incident note.

Knowledge check

77. Does cherry-pick preserve the original commit ID?

78. Why is bisect logarithmically efficient?

79. Why should blame not be used to assign personal fault?

Completion evidence: _____

Unit 14: Git Ignore, Attributes, Line Endings and Binary Files

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Control untracked-file exclusions.
- Normalize file behavior.
- Handle large and binary files responsibly.

Why this matters in DevOps

Cross-platform repositories break when generated files, secrets, line endings and binary assets are unmanaged.

.gitignore

Ignore rules affect untracked files. They do not automatically remove files already committed.

.gitattributes

Attributes control text normalization, diff behavior, merge drivers and export behavior.

Large files

Git LFS stores lightweight pointers in Git while large content is held in an associated object service.

Command laboratory

Test ignore rules

```
git check-ignore -v path/to/file
```

The command identifies the matching rule.

Stop tracking ignored file

```
git rm --cached path/to/file
```

The local file remains while the next commit removes it from the repository.

Example attributes

```
printf '* text=auto\n*.sh text eol=lf\n*.bat text eol=crlf\n' > .gitattributes
```

Define platform expectations explicitly.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Hands-on lab

80. Build a .gitignore for logs, local environment files and build output.
81. Verify rules with check-ignore.
82. Add attributes for shell and Windows batch files.

Knowledge check

83. Why can an ignored file remain tracked?
84. What problem does text=auto address?
85. When is Git LFS appropriate?

Completion evidence: _____

Unit 15: Submodules, Subtrees and Monorepos

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Compare repository composition strategies.
- Use submodules safely.
- Identify monorepo tradeoffs.

Why this matters in DevOps

Large organizations must decide how multiple components and dependencies are versioned and released.

Submodules

A submodule records a specific commit of another repository. Clones require explicit initialization and updates.

Subtree

Subtree workflows copy another repository's content into a subdirectory while preserving a synchronization mechanism.

Monorepo

A monorepo keeps many projects in one repository, simplifying atomic cross-project changes but increasing tooling and scale demands.

Command laboratory

Add submodule

```
git submodule add <repository-url> vendor/component
git commit -m "build: add component submodule"
```

The parent repository records a gitlink and .gitmodules.

Clone recursively

```
git clone --recurse-submodules <repository-url>
```

Existing clones can use submodule update --init --recursive.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Hands-on lab

86. Create two local repositories and attach one as a submodule to the other.
87. Clone the parent into a new directory recursively.
88. Document when a monorepo would be preferable.

Knowledge check

89. Does a submodule automatically follow the latest remote commit?
90. What does the parent repository store for a submodule?
91. Name one monorepo advantage and one cost.

Completion evidence: _____

Unit 16: Git Hooks, Aliases and Automation

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Create local hooks.
- Understand client-side versus server-side enforcement.
- Use aliases without hiding critical behavior.

Why this matters in DevOps

Automation shifts quality checks earlier, but local controls alone cannot enforce organization-wide policy.

Hooks

Git invokes executable scripts for events such as pre-commit, commit-msg and pre-push. Native hooks are not cloned automatically.

Policy enforcement

CI and server-side rules are authoritative. Local hooks provide fast feedback.

Aliases

Aliases reduce repetitive typing, but team documentation should still use understandable standard commands.

Command laboratory

Create alias

```
git config --global alias.lg "log --graph --decorate --oneline --all"
```

Use git lg to inspect history.

Simple pre-commit hook

```
mkdir -p .git/hooks
cat > .git/hooks/pre-commit <<'EOF'
#!/usr/bin/env bash
set -e
if git diff --cached --check; then
    exit 0
fi
EOF
chmod +x .git/hooks/pre-commit
```

This rejects whitespace errors in staged content.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Hands-on lab

92. Install and test the alias.
93. Create a hook that blocks a sample forbidden string.
94. Explain how CI duplicates or strengthens the rule.

Knowledge check

95. Why are local hooks insufficient for compliance?

96. Are .git/hooks files committed by default?

97. What risk comes from overly complex aliases?

Completion evidence: _____

Unit 17: Git Security, Signing and Secret Prevention

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Protect credentials.
- Understand signed commits and tags.
- Respond to an exposed secret.

Why this matters in DevOps

Git history is durable and widely replicated. A committed secret must be treated as compromised even if later deleted.

Secret prevention

Use secret managers, environment injection and automated secret scanning. Never commit private keys, tokens or production passwords.

Signing

Cryptographic signatures can establish that a commit or tag was signed by a controlled identity. Verification policy matters more than simply displaying a badge.

Incident response

Revoke or rotate the secret first, assess exposure, then remove sensitive history if required and coordinate repository cleanup.

Command laboratory

Inspect signatures

```
git log --show-signature
git tag -v v1.0.0
```

Verification requires trusted key configuration.

Search tracked content

```
git grep -n -I -E 'password|secret|token'
```

This is only a simple local check, not a complete scanner.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Hands-on lab

98. Create a dummy secret pattern and ensure your local check catches it before commit.
99. Write an exposed-token response checklist.
100. Enable multi-factor authentication on your Git hosting account.

Knowledge check

101. Why is deleting the latest file insufficient?
102. What must happen before history rewriting?
103. What does a valid signature prove and not prove?

Completion evidence: _____

Unit 18: Repository Maintenance and Performance

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Inspect repository size.
- Understand packfiles and garbage collection.
- Maintain large repositories carefully.

Why this matters in DevOps

Repository health affects clone speed, CI duration, developer productivity and storage cost.

Packfiles

Git compresses and delta-encodes objects into packfiles.

Garbage collection

git gc consolidates objects and prunes unreachable data according to retention rules.

Scale controls

Shallow clones, partial clones, sparse checkout and caching can reduce transfer and checkout costs, but each has functional tradeoffs.

Command laboratory

Count objects

```
git count-objects -vH
```

Inspect loose objects and pack size.

Maintenance

```
git maintenance run
```

```
git gc
```

Avoid aggressive cleanup during uncertain recovery situations.

Sparse checkout

```
git sparse-checkout init --cone
```

```
git sparse-checkout set service-a docs
```

Use when only part of a large working tree is needed.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Hands-on lab

104. Inspect object statistics before and after several commits.
105. Run maintenance on a disposable repository.
106. Test sparse checkout in a repository with multiple directories.

Knowledge check

107. What is a packfile?
108. Why can aggressive pruning reduce recovery options?
109. What is the difference between shallow and sparse?

Completion evidence: _____

Unit 19: GitHub Foundations: Accounts, Repositories and Organizations

Completion status: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Navigate GitHub repository features.
- Explain organizations and teams.
- Apply secure account settings.

Why this matters in DevOps

GitHub adds collaboration, governance, automation and integrations around Git repositories.

Repository services

GitHub provides issues, pull requests, Actions, releases, code search, security scanning, discussions, projects and APIs.

Organizations

Organizations centralize ownership, identity, teams, policies and billing.

Authentication

Use multi-factor authentication, least-privilege tokens or approved SSH keys, and organization SSO where required.

Command laboratory

GitHub CLI login

```
gh auth login
```

```
gh auth status
```

Use the official CLI only in a trusted environment.

Create repository

```
gh repo create volume3-github-lab --private --source=. --remote=origin --push
```

Review visibility before creation.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Hands-on lab

110. Create a private practice repository.
111. Add a description, README and license decision.
112. Review account security and active sessions.

Knowledge check

113. What is the difference between a user repository and organization repository?
114. Why should tokens be scoped?
115. Which features exist beyond raw Git hosting?

Completion evidence: _____

Unit 20: Pull Requests and Professional Code Review

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Open a high-quality pull request.
- Review code constructively.
- Respond to feedback with traceability.

Why this matters in DevOps

Pull requests are controlled integration records that combine discussion, checks, approvals and merge decisions.

Pull-request quality

Describe the problem, solution, testing, risk, rollout and rollback. Keep scope reviewable.

Review method

Check correctness, security, operability, test coverage, maintainability and consistency. Comment on code and reasoning, not the person.

Feedback lifecycle

Push follow-up commits, reply with evidence, and resolve threads only when the concern is addressed.

Command laboratory

Create pull request

```
gh pr create --fill --base main --head feature/network-docs
```

Edit generated content to include risk and testing.

Inspect checks

```
gh pr checks
```

```
gh pr view --comments
```

Do not merge with unexplained failures.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Hands-on lab

116. Open a pull request from your practice branch.
117. Ask a peer or use a second account/repository simulation for review.
118. Respond to three review comments with commits and explanations.

Knowledge check

119. What information belongs in a pull request description?
120. What makes review feedback constructive?
121. Who should resolve a review thread?

Completion evidence: _____

Unit 21: Branch Protection, Rulesets and Governance

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Design protected-branch controls.
- Explain status checks and approval policies.
- Balance safety with delivery speed.

Why this matters in DevOps

Governance prevents accidental bypass of review, testing and audit requirements.

Typical controls

Required pull requests, required approving reviews, status checks, conversation resolution, signed commits, linear history and restricted force pushes.

Rulesets

Rules can target branches or tags by pattern and can apply organization-wide.

Emergency process

A break-glass path must be limited, logged, reviewed afterward and tested in advance.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Hands-on lab

122. Write a main-branch ruleset for a production repository.
123. Define required checks and code-owner approval.
124. Document an emergency override process.

Knowledge check

125. Why are protected branches useful?
126. What risk comes from too many unreliable required checks?
127. What must follow a break-glass action?

Completion evidence: _____

Unit 22: GitHub Issues, Projects, Discussions and Documentation

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Track work effectively.
- Connect issues to pull requests.
- Use documentation as an operational asset.

Why this matters in DevOps

DevOps delivery needs visible prioritization, ownership and decisions, not only code.

Issues

A strong issue includes context, acceptance criteria, constraints, owner and priority.

Projects

Project boards and tables provide planning views over issues and pull requests.

Documentation

README, contributing guide, architecture decisions, runbooks and support ownership reduce operational risk.

Command laboratory

Create issue

```
gh issue create --title "Document DNS troubleshooting" --body "Acceptance criteria: ..."
```

Use concrete completion criteria.

Link closure

```
git commit -m "docs: add DNS runbook\n\nCloses #1"
```

The platform can close linked issues when merged to the default branch.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Hands-on lab

128. Create an issue template.
129. Create a small project board.
130. Add README, CONTRIBUTING and an operations runbook to your practice repository.

Knowledge check

131. What distinguishes an issue from a pull request?
132. Why are acceptance criteria important?
133. Which documentation helps production support?

Completion evidence: _____

Unit 23: GitHub Actions Foundations

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Explain workflow structure.
- Trigger a basic CI job.
- Protect secrets and permissions.

Why this matters in DevOps

GitHub Actions turns repository events into automated validation and delivery workflows.

Workflow structure

A YAML workflow defines triggers, jobs, runners, steps and permissions.

Security model

Workflows can access repository tokens and secrets. Pin trusted actions, minimize permissions and treat pull requests from forks carefully.

Artifacts and caches

Artifacts preserve outputs; caches accelerate reusable dependencies. They serve different purposes.

Command laboratory

Basic workflow

```
mkdir -p .github/workflows
cat > .github/workflows/ci.yml <<'EOF'
name: CI
on:
  pull_request:
  push:
    branches: [main]
permissions:
  contents: read
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: git diff --check
EOF
```

Commit and push the workflow, then inspect its run.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Hands-on lab

134. Add the workflow to a practice repository.

- 135. Create a pull request that passes.
- 136. Deliberately add a whitespace error and observe failure.

Knowledge check

- 137. What is a runner?
- 138. Why should workflow permissions be minimized?
- 139. How is an artifact different from a cache?

Completion evidence: _____

Unit 24: GitHub Releases, Packages and Environments

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Create controlled releases.
- Understand deployment environments.
- Connect approvals to delivery.

Why this matters in DevOps

Release management must preserve provenance and restrict sensitive deployment targets.

Releases

A GitHub Release associates notes and assets with a tag.

Packages

Container images and other packages can be stored with repository-linked permissions.

Environments

Environments can protect secrets and require reviewers or waiting periods before deployment jobs continue.

Command laboratory

Create release draft

```
gh release create v0.1.0 --draft --generate-notes
```

Review generated notes before publishing.

List releases

```
gh release list
```

Validate tags and release state.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Hands-on lab

140. Create a draft release.
141. Attach a small checksum file as an asset.
142. Design staging and production environment protection rules.

Knowledge check

143. Does a GitHub Release replace a Git tag?
144. Why are environment-specific secrets useful?
145. What is release provenance?

Completion evidence: _____

Unit 25: Enterprise GitHub Operations and API Automation

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Understand enterprise controls.
- Automate safely with APIs.
- Design repository standards.

Why this matters in DevOps

At scale, consistent governance and inventory cannot be maintained manually.

Enterprise controls

Identity federation, SSO, audit logs, organization policies, approved applications, runner governance and data retention.

API automation

REST, GraphQL and GitHub CLI can inventory repositories, open issues, manage settings and report policy drift.

Platform engineering

Internal templates and reusable workflows provide paved roads that make the safe path easier.

Command laboratory

Query repository

```
gh api repos/{owner}/{repo} --jq '{name,visibility,default_branch}'
```

Replace placeholders or run inside a repository with appropriate arguments.

List workflows

```
gh workflow list
```

Use read-only inspection before changing settings.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Hands-on lab

146. Define a standard repository baseline.
147. Write a read-only inventory script using gh.
148. Create a platform checklist for onboarding a new service.

Knowledge check

149. What belongs in an enterprise repository baseline?
150. Why start API automation read-only?
151. What is a paved road?

Completion evidence: _____

Unit 26: Networking Fundamentals and the OSI/TCP-IP Models

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Explain layered networking.
- Map common protocols to layers.
- Use encapsulation terminology.

Why this matters in DevOps

DevOps engineers troubleshoot systems that span applications, transport, routing, links and physical infrastructure.

Layering

Layers separate responsibilities. The OSI model is conceptual; the practical Internet stack is often described as link, internet, transport and application.

Encapsulation

Application data receives transport, IP and link headers as it moves downward. The receiver removes them upward.

Troubleshooting method

Start with scope and symptoms, then move layer by layer instead of guessing.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Common mistakes and prevention

- Do not run destructive or security-sensitive commands on production systems without authorization and rollback.
- Do not treat a successful command as proof that the full service works.
- Record exact evidence rather than relying on memory.

Hands-on lab

152. Draw both models and map Ethernet, IP, TCP, UDP, TLS, HTTP and DNS.
153. For a web request, describe encapsulation.
154. Write one failure example for each layer.

Knowledge check

155. Is TLS an application or transport protocol in every model?
156. Why are models still useful?
157. What does encapsulation mean?

Completion evidence: _____

Unit 27: Ethernet, MAC Addresses, Switching and ARP

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Explain local network delivery.
- Understand switching tables.
- Inspect ARP or neighbor state.

Why this matters in DevOps

Many apparent IP failures are actually local link or neighbor-resolution problems.

Ethernet

Frames carry source and destination MAC addresses on a local link.

Switching

A switch learns source MAC locations and forwards frames toward the known destination port or floods when unknown.

ARP and NDP

IPv4 ARP resolves local IP addresses to MAC addresses. IPv6 uses Neighbor Discovery rather than ARP.

Command laboratory

Inspect links

```
ip link
ip address
```

Identify interface state and addresses.

Inspect neighbors

```
ip neigh
```

Entries may be reachable, stale, failed or incomplete.

Capture ARP

```
sudo tcpdump -ni any arp
```

Use only on systems and networks you are authorized to inspect.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Common mistakes and prevention

- Do not run destructive or security-sensitive commands on production systems without authorization and rollback.
- Do not treat a successful command as proof that the full service works.
- Record exact evidence rather than relying on memory.

Hands-on lab

158. Inspect your interface and neighbor table.
159. Ping a local gateway and watch neighbor-state changes.
160. Capture an authorized ARP exchange.

Knowledge check

- 161. Does a router forward based on MAC across the Internet?
- 162. What does an incomplete neighbor entry suggest?
- 163. How does a switch learn ports?

Completion evidence: _____

Unit 28: IPv4 Addressing, Subnets and CIDR

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Read IPv4 addresses and prefixes.
- Calculate network ranges.
- Recognize private and public address space.

Why this matters in DevOps

Cloud networks, firewall rules and Kubernetes clusters require accurate subnet planning.

Address and prefix

A CIDR prefix such as /24 identifies how many leading bits are the network portion.

Subnet properties

A traditional subnet has a network address, usable host range and broadcast address. Cloud providers may reserve additional addresses.

Private ranges

Common private IPv4 ranges are 10.0.0.0/8, 172.16.0.0/12 and 192.168.0.0/16.

Command laboratory

Inspect route and address

```
ip -4 addr
ip -4 route
```

Identify local prefixes and default route.

Calculate with Python

```
python3 - <<'PY'
import ipaddress
n=ipaddress.ip_network('10.20.30.0/27')
print(n.network_address, n.broadcast_address, n.num_addresses)
print(list(n.hosts())[0], list(n.hosts())[-1])
PY
```

Use reliable tooling for production calculations.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Common mistakes and prevention

- Do not run destructive or security-sensitive commands on production systems without authorization and rollback.
- Do not treat a successful command as proof that the full service works.
- Record exact evidence rather than relying on memory.

Hands-on lab

164. Calculate /24, /27 and /30 examples.

165. Design three non-overlapping application subnets.
166. Explain why overlapping CIDRs break network integration.

Knowledge check

167. How many total addresses are in /27?
168. What is the prefix of 255.255.255.0?
169. Why should cloud CIDRs avoid overlap?

Completion evidence: _____

Unit 29: IPv6 Fundamentals

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Recognize IPv6 notation.
- Explain address scopes.
- Use IPv6-aware troubleshooting.

Why this matters in DevOps

Modern networks and cloud platforms increasingly require dual-stack competence.

Address size and notation

IPv6 uses 128-bit addresses written in hexadecimal groups. One run of zeros can be compressed with ::.

Scopes

Link-local addresses commonly begin fe80::. Global unicast commonly falls within 2000::/3. ::1 is loopback.

Neighbor discovery

IPv6 uses ICMPv6 for neighbor discovery and other essential control functions.

Command laboratory

Inspect IPv6

```
ip -6 addr
ip -6 route
```

Check whether the host is IPv6-enabled.

Ping IPv6 loopback

```
ping -6 ::1
```

This validates local IPv6 stack behavior.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Common mistakes and prevention

- Do not run destructive or security-sensitive commands on production systems without authorization and rollback.
- Do not treat a successful command as proof that the full service works.
- Record exact evidence rather than relying on memory.

Hands-on lab

170. Expand and compress sample IPv6 addresses.
171. Identify link-local and loopback addresses.
172. Document whether your lab supports IPv6 and why.

Knowledge check

173. How many bits are in IPv6?
174. What replaces ARP?
175. Why can blocking all ICMPv6 cause failures?

Completion evidence: _____

Unit 30: Routing, Gateways, NAT and ICMP

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Read a routing table.
- Explain longest-prefix match.
- Understand NAT and diagnostic ICMP.

Why this matters in DevOps

Packets reach remote networks through routing decisions, not simply through DNS or application configuration.

Routing

The host selects the most specific matching route. A default route handles destinations without a more specific entry.

NAT

Network address translation rewrites address or port information, commonly allowing private hosts to share public connectivity.

ICMP

ICMP carries control and error information and supports diagnostics such as ping and traceroute behavior.

Command laboratory

Inspect routes

```
ip route get 8.8.8.8
ip route
```

The get command shows the selected path.

Trace path

```
traceroute example.com
tracert example.com
```

Availability varies by operating system.

Ping with count

```
ping -c 4 8.8.8.8
```

Packet loss or latency does not by itself identify the failing layer.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Common mistakes and prevention

- Do not run destructive or security-sensitive commands on production systems without authorization and rollback.
- Do not treat a successful command as proof that the full service works.
- Record exact evidence rather than relying on memory.

Hands-on lab

- 176. Predict the route for three destinations.
- 177. Compare ping to gateway, public IP and hostname.
- 178. Document NAT in your home or lab network.

Knowledge check

- 179. What is longest-prefix match?
- 180. What does a default route represent?
- 181. Why can ping fail while HTTPS still works?

Completion evidence: _____

Unit 31: TCP and UDP

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Explain connections and ports.
- Understand TCP reliability mechanisms.
- Choose TCP or UDP based on requirements.

Why this matters in DevOps

Application behavior depends heavily on transport semantics.

TCP

TCP establishes a connection, orders bytes, retransmits losses and controls flow and congestion.

UDP

UDP sends independent datagrams without connection establishment or built-in delivery guarantees.

Ports and sockets

A network conversation is identified by protocol and endpoint address/port combinations.

Command laboratory

Inspect listening sockets

```
ss -lntup
```

Privileges may be needed to show process details.

Inspect TCP connections

```
ss -tn state established
```

Observe local and peer endpoints.

Create test listener

```
python3 -m http.server 8000
```

Run in a lab and inspect the listening socket.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Common mistakes and prevention

- Do not run destructive or security-sensitive commands on production systems without authorization and rollback.
- Do not treat a successful command as proof that the full service works.
- Record exact evidence rather than relying on memory.

Hands-on lab

182. Run a temporary web server.
183. Connect with curl and inspect sockets.
184. Compare a DNS UDP exchange and an HTTPS TCP connection using authorized capture tools.

Knowledge check

- 185. Does UDP mean unreliable application?
- 186. What is the purpose of the TCP handshake?
- 187. Can two processes listen on the same port and address normally?

Completion evidence: _____

Unit 32: DNS Architecture and Troubleshooting

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Explain recursive and authoritative DNS.
- Query records directly.
- Troubleshoot resolution systematically.

Why this matters in DevOps

DNS failures appear as application outages even when hosts and ports are healthy.

Resolution chain

A stub resolver asks a recursive resolver, which may query root, TLD and authoritative servers and cache results.

Record types

A and AAAA map names to addresses; CNAME aliases names; MX directs mail; TXT stores text; NS delegates zones; SOA describes zone authority.

Caching

TTL controls how long answers may be cached. Negative answers can also be cached.

Command laboratory

Query DNS

```
dig example.com A
dig example.com AAAA
dig +trace example.com
```

Trace demonstrates delegation but may be filtered.

Specify resolver

```
dig @1.1.1.1 example.com A
```

Compare results only where external resolvers are permitted.

Inspect host configuration

```
cat /etc/resolv.conf
resolvectl status
```

The active resolver depends on the system.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Common mistakes and prevention

- Do not run destructive or security-sensitive commands on production systems without authorization and rollback.
- Do not treat a successful command as proof that the full service works.
- Record exact evidence rather than relying on memory.

Hands-on lab

- 188. Query A, AAAA, NS, MX and TXT records.
- 189. Compare local and public resolver answers.
- 190. Create a DNS incident checklist.

Knowledge check

- 191. What is the difference between recursive and authoritative servers?
- 192. What does TTL affect?
- 193. Why can one client resolve while another cannot?

Completion evidence: _____

Unit 33: HTTP, HTTPS and API Networking

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Read HTTP requests and responses.
- Understand methods and status codes.
- Troubleshoot APIs with curl.

Why this matters in DevOps

Most DevOps delivery and observability systems communicate through HTTP APIs.

HTTP message model

A request contains method, target, headers and optional body. A response contains status, headers and optional body.

Status classes

2xx success, 3xx redirection, 4xx client-side request issues, 5xx server-side failure.

Idempotency

GET, PUT and DELETE are defined as idempotent in semantics, though implementations can violate expectations. POST is not generally idempotent.

Command laboratory

Inspect headers

```
curl -I https://example.com
curl -v https://example.com/
```

Verbose output reveals DNS, connection, TLS and protocol details.

API request

```
curl -sS -H 'Accept: application/json' https://api.github.com | head
```

Respect API limits and terms.

Timing

```
curl -sS -o /dev/null -w 'dns=%{time_namelookup} connect=%{time_connect} tls=%{time_appconnect}
total=%{time_total}\n' https://example.com
```

Timing separates stages of request latency.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Common mistakes and prevention

- Do not run destructive or security-sensitive commands on production systems without authorization and rollback.
- Do not treat a successful command as proof that the full service works.
- Record exact evidence rather than relying on memory.

Hands-on lab

194. Inspect a redirect chain.

- 195. Record DNS, connect, TLS and total timings.
- 196. Send a local GET and POST request to a test service.

Knowledge check

- 197. What does 502 usually indicate?
- 198. What is idempotency?
- 199. Why is curl -v useful?

Completion evidence: _____

Unit 34: TLS, Certificates and Public Key Infrastructure

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Explain the TLS handshake conceptually.
- Inspect certificates.
- Recognize common certificate failures.

Why this matters in DevOps

TLS protects service identity and data in transit, but only when trust, hostname and validity checks succeed.

Certificate chain

A server certificate is validated through intermediates to a trusted root. The client also checks hostname, validity period and intended usage.

Handshake

The peers negotiate protocol and cryptographic parameters, authenticate the server and establish shared session keys.

Common failures

Expired certificate, wrong hostname, missing intermediate, untrusted CA, clock skew or protocol mismatch.

Command laboratory

Inspect endpoint

```
openssl s_client -connect example.com:443 -servername example.com </dev/null
```

SN1 is supplied with -servername.

Show certificate fields

```
echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/null | openssl x509 -noout -subject -issuer -dates -ext subjectAltName
```

Inspect identity and validity.

Curl verification

```
curl -v https://example.com/
```

Do not use -k as a permanent fix.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Common mistakes and prevention

- Do not run destructive or security-sensitive commands on production systems without authorization and rollback.
- Do not treat a successful command as proof that the full service works.
- Record exact evidence rather than relying on memory.

Hands-on lab

200. Inspect one public certificate.

- 201. Identify subject alternative names and expiry.
- 202. Write a certificate-renewal runbook.

Knowledge check

- 203. What hostname field is normally checked?
- 204. Why is a missing intermediate a problem?
- 205. What risk does curl -k introduce?

Completion evidence: _____

Unit 35: DHCP, Time, Proxies, VPNs and CDNs

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Understand supporting network services.
- Explain forward and reverse proxies.
- Recognize delivery-path intermediaries.

Why this matters in DevOps

Real application traffic passes through many infrastructure services beyond routers and servers.

DHCP

DHCP supplies addresses and options such as gateway and DNS to clients.

Time synchronization

Accurate time is required for TLS validation, logs, authentication and distributed systems.

Proxies

A forward proxy acts for clients. A reverse proxy acts in front of servers.

VPN and CDN

VPNs create protected network connectivity. CDNs cache and serve content closer to users and absorb traffic at the edge.

Command laboratory

Inspect time

```
timedatectl status
```

Verify synchronization status.

Inspect proxy environment

```
env | grep -i proxy
```

Proxy variables can alter CLI behavior.

Inspect headers

```
curl -I https://example.com
```

Headers may reveal caching or proxy behavior.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Common mistakes and prevention

- Do not run destructive or security-sensitive commands on production systems without authorization and rollback.
- Do not treat a successful command as proof that the full service works.
- Record exact evidence rather than relying on memory.

Hands-on lab

206. Document your system's DHCP, DNS and time sources.

- 207. Draw client -> CDN -> load balancer -> reverse proxy -> service.
- 208. List symptoms caused by clock drift.

Knowledge check

- 209. What is the difference between forward and reverse proxy?
- 210. Why does time affect TLS?
- 211. What can a CDN cache?

Completion evidence: _____

Unit 36: Linux Network Administration Commands

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Use the modern ip and ss tools.
- Test paths and ports.
- Collect repeatable diagnostic evidence.

Why this matters in DevOps

Command fluency enables quick, evidence-based incident response.

Interface and route tools

ip replaces many older ifconfig and route use cases.

Socket tools

ss provides listening and connection state.

Testing tools

ping tests ICMP reachability, traceroute estimates path, nc tests basic TCP/UDP connectivity, and curl tests application behavior.

Command laboratory

Baseline

```
hostnamectl  
ip -br addr  
ip route  
ss -lntup
```

Capture a compact host network baseline.

Port test

```
nc -vz example.com 443
```

Use only for authorized destinations.

DNS and HTTP

```
dig example.com  
curl -v https://example.com/
```

Test progressively from name to application.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Common mistakes and prevention

- Do not run destructive or security-sensitive commands on production systems without authorization and rollback.
- Do not treat a successful command as proof that the full service works.
- Record exact evidence rather than relying on memory.

Hands-on lab

- 212. Create a one-command baseline report.
- 213. Test a known open and closed local port.
- 214. Write a decision tree using ip, dig, nc, curl and ss.

Knowledge check

- 215. Why prefer ip over ifconfig?
- 216. What does ss -lnt mean?
- 217. Why should tests progress layer by layer?

Completion evidence: _____

Unit 37: Packet Capture with tcpdump and Wireshark

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Capture authorized traffic.
- Apply useful filters.
- Interpret packet sequences.

Why this matters in DevOps

Packet capture provides direct evidence when logs and application messages are insufficient.

Capture discipline

Capture only authorized traffic, minimize sensitive data, set file limits and store captures securely.

Filters

Capture filters reduce collected traffic. Display filters in Wireshark narrow analysis after capture.

Interpretation

Look for DNS answers, SYN/SYN-ACK patterns, retransmissions, resets, TLS alerts and HTTP behavior where visible.

Command laboratory

DNS capture

```
sudo tcpdump -ni any port 53
```

Stop after a short authorized test.

HTTP test capture

```
sudo tcpdump -ni any 'tcp port 8000' -w lab.pcap
```

Open the pcap in Wireshark.

Read file

```
tcpdump -nn -r lab.pcap
```

-nn avoids name and service resolution.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Common mistakes and prevention

- Do not run destructive or security-sensitive commands on production systems without authorization and rollback.
- Do not treat a successful command as proof that the full service works.
- Record exact evidence rather than relying on memory.

Hands-on lab

218. Capture a local HTTP request.
219. Identify handshake packets.
220. Capture a DNS lookup and locate query and response.

Knowledge check

- 221. What is the difference between capture and display filters?
- 222. Why use -nn?
- 223. What indicates a TCP reset?

Completion evidence: _____

Unit 38: Firewalls and Network Policy

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Understand stateful filtering.
- Inspect Linux firewall frameworks.
- Design least-privilege rules.

Why this matters in DevOps

Firewalls reduce attack surface and control service communication.

Stateful filtering

A stateful firewall tracks connection state and can allow replies to established flows.

Linux frameworks

nftables is the modern Linux framework; iptables may be a compatibility interface depending on the distribution.

Policy design

Default deny, explicit required flows, documented ownership, logging with rate limits and tested rollback.

Command laboratory

Inspect nftables

```
sudo nft list ruleset
```

Do not change remote-host firewall rules without console recovery.

Inspect iptables compatibility

```
sudo iptables -S
```

Output depends on the host.

Local service test

```
ss -lntp
```

```
nc -vz 127.0.0.1 8000
```

A listening service and firewall permission are separate conditions.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Common mistakes and prevention

- Do not run destructive or security-sensitive commands on production systems without authorization and rollback.
- Do not treat a successful command as proof that the full service works.
- Record exact evidence rather than relying on memory.

Hands-on lab

224. Design rules for a web server with SSH administration.
225. Document source, destination, protocol, port, owner and justification.
226. Test rules only in a disposable VM.

Knowledge check

- 227. What is default deny?
- 228. Why can a service listen but remain unreachable?
- 229. Why is rollback planning essential for firewall changes?

Completion evidence: _____

Unit 39: Load Balancers, Reverse Proxies and Service Discovery

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Explain load-balancing layers.
- Understand health checks.
- Recognize service-discovery patterns.

Why this matters in DevOps

Production services scale and survive failure through controlled traffic distribution.

Layer 4 and Layer 7

Layer 4 load balancing uses transport information. Layer 7 understands application protocols such as HTTP.

Health checks

Checks should indicate whether an instance can safely serve traffic, not merely whether a process exists.

Discovery

Clients or proxies locate service instances through DNS, registries, platform APIs or sidecars.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Common mistakes and prevention

- Do not run destructive or security-sensitive commands on production systems without authorization and rollback.
- Do not treat a successful command as proof that the full service works.
- Record exact evidence rather than relying on memory.

Hands-on lab

230. Draw a three-tier architecture with external and internal load balancers.
231. Define liveness, readiness and dependency-aware health checks.
232. Explain how stale discovery data could cause failures.

Knowledge check

233. What is the practical difference between L4 and L7 balancing?
234. Why should readiness differ from liveness?
235. What is service discovery?

Completion evidence: _____

Unit 40: Docker and Kubernetes Networking Foundations

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Explain container network namespaces.
- Understand Docker bridge networking.
- Describe Kubernetes pod and service networking.

Why this matters in DevOps

Containers introduce virtual interfaces, namespaces, overlays and platform-managed service discovery.

Containers

A container commonly receives its own network namespace connected through a virtual Ethernet pair to a bridge or other driver.

Docker

Bridge, host, none and overlay are common modes. Port publishing maps host traffic to container ports.

Kubernetes

Each pod generally receives an IP. Services provide stable virtual access, while the CNI plugin implements pod networking. Ingress or Gateway APIs manage application entry.

Command laboratory

Docker inspect

```
docker network ls
docker network inspect bridge
```

Requires Docker in the lab.

Container test

```
docker run --rm -d --name web -p 8080:80 nginx
docker port web
curl http://127.0.0.1:8080
docker rm -f web
```

Use an official image in an isolated lab.

Kubernetes inspection

```
kubectl get pods -o wide
kubectl get svc
kubectl get endpointslices
```

Requires a Kubernetes lab.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Common mistakes and prevention

- Do not run destructive or security-sensitive commands on production systems without authorization and rollback.

- Do not treat a successful command as proof that the full service works.
- Record exact evidence rather than relying on memory.

Hands-on lab

236. Run two containers on a user-defined bridge and test name resolution.
237. Publish one port and inspect NAT behavior.
238. Draw pod -> service -> ingress traffic flow.

Knowledge check

239. Does a container port automatically become a host port?
240. What does a Kubernetes Service provide?
241. What component implements pod networking?

Completion evidence: _____

Unit 41: Cloud Networking Foundations

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Understand VPC/VNet concepts.
- Design public and private subnets.
- Compare security groups and network ACL concepts.

Why this matters in DevOps

Every major cloud platform exposes software-defined networking that must be planned for availability and security.

Virtual networks

AWS VPC, Azure VNet and Google Cloud VPC provide logically isolated address spaces and routing.

Subnets and routes

Public reachability requires more than a public label: route paths, gateways, addresses and security controls must align.

Security controls

Cloud firewalls commonly operate at instance, interface, subnet or policy levels with provider-specific statefulness.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Common mistakes and prevention

- Do not run destructive or security-sensitive commands on production systems without authorization and rollback.
- Do not treat a successful command as proof that the full service works.
- Record exact evidence rather than relying on memory.

Hands-on lab

242. Design a two-availability-zone web architecture.
243. Allocate non-overlapping CIDRs for web, application and database tiers.
244. List required flows and deny everything else.

Knowledge check

245. What makes a subnet public?
246. Why are non-overlapping ranges important?
247. Are all cloud firewall constructs stateful?

Completion evidence: _____

Unit 42: Network Security and Zero Trust

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Apply least privilege to communication.
- Understand segmentation and identity-aware access.
- Protect management planes.

Why this matters in DevOps

Network location alone is insufficient proof of trust in modern distributed environments.

Segmentation

Separate environments and sensitivity levels, minimize east-west access and restrict administrative paths.

Zero trust principles

Verify explicitly, use least privilege and assume breach. Identity, device posture, workload identity and policy complement network controls.

Management security

Use bastions or approved access brokers, strong authentication, short-lived credentials, logging and controlled egress.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Common mistakes and prevention

- Do not run destructive or security-sensitive commands on production systems without authorization and rollback.
- Do not treat a successful command as proof that the full service works.
- Record exact evidence rather than relying on memory.

Hands-on lab

248. Create a communication matrix for a three-tier service.
249. Replace broad CIDR access with identity-aware or narrowly scoped rules where possible.
250. Design an administrator access path without public SSH to every host.

Knowledge check

251. What does assume breach mean?
252. Why is a flat network risky?
253. What is east-west traffic?

Completion evidence: _____

Unit 43: Systematic Network Troubleshooting and Incident Response

Completion status: ☐ Not started ☐ In progress ☐ Completed Date: _____

Learning objectives

- Use a repeatable troubleshooting framework.
- Separate symptom from cause.
- Write an operational incident timeline.

Why this matters in DevOps

Fast troubleshooting comes from disciplined evidence collection, not random command execution.

Framework

Confirm scope, timeline and recent changes. Reproduce safely. Test local process, listener, firewall, route, DNS, transport, TLS and application layers. Compare healthy and unhealthy paths.

Evidence

Record exact timestamps, commands, outputs, affected identities and changes. Preserve volatile evidence when appropriate.

Recovery

Prefer reversible mitigation, validate user impact, monitor stability and perform root-cause analysis after restoration.

Command laboratory

Baseline sequence

```
date -Is
hostname
ip -br addr
ip route
ss -lntup
resolvectl status 2>/dev/null || cat /etc/resolv.conf
curl -vk --connect-timeout 5 https://example.com/
```

Remove -k when validating trust; it appears here only to separate connectivity from certificate validation during a controlled diagnostic.

Path-specific route

```
ip route get <destination-ip>
```

Replace the placeholder.

Targeted capture

```
sudo tcpdump -ni any host <destination-ip> and port 443
```

Capture narrowly and only with authorization.

How companies use this

In professional environments, this topic is governed through documented standards, peer review, automation, least privilege and measurable validation.

Common mistakes and prevention

- Do not run destructive or security-sensitive commands on production systems without authorization and rollback.
- Do not treat a successful command as proof that the full service works.
- Record exact evidence rather than relying on memory.

Hands-on lab

254. Simulate a local service outage.
255. Write a timeline from detection to validation.
256. Create a runbook with decision points and escalation criteria.

Knowledge check

257. Why must recent changes be checked early?
258. What is the difference between mitigation and root cause?
259. Why compare healthy and unhealthy clients?

Completion evidence: _____

Unit 44: Capstone 1: Enterprise Git Collaboration

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Build a governed repository workflow.
- Demonstrate branch, review and release controls.
- Produce auditable documentation.

Why this matters in DevOps

This capstone integrates professional Git and GitHub practices.

Scenario

A small platform team maintains deployment scripts and network runbooks. Main is protected, releases are tagged, changes require checks and review, and secrets are prohibited.

Required deliverables

- Repository baseline and README.
- Issue and pull-request templates.
- Branch protection design.
- CI validation workflow.
- Release and rollback procedure.
- Contributor and security guidance.

Hands-on lab

260. Create the repository and documentation.
261. Open issues for planned work.
262. Implement two features through branches and pull requests.
263. Create and test CI checks.
264. Tag a release and produce release notes.
265. Perform a controlled revert exercise.

Knowledge check

266. Can another engineer reproduce your workflow?
267. Is every release traceable?
268. Are emergency and rollback paths documented?

Completion evidence: _____

Unit 45: Capstone 2: Network Architecture and Troubleshooting Lab

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Design a production-style network.
- Validate paths methodically.
- Produce diagrams and runbooks.

Why this matters in DevOps

This capstone integrates addressing, routing, DNS, TLS, load balancing, firewalls and incident handling.

Scenario

Design a public web tier, private application tier and isolated data tier across at least two failure domains. Administrative access must be controlled and logged.

Required deliverables

- CIDR plan.
- Traffic matrix.
- DNS and certificate plan.
- Health-check definitions.
- Monitoring and logging signals.
- Troubleshooting runbook.
- Failure simulation report.

Hands-on lab

269. Draw the architecture.
270. Calculate subnet ranges.
271. Define route and firewall requirements.
272. Deploy a simplified local or cloud lab.
273. Simulate DNS, port and certificate failures.
274. Document diagnosis and recovery.

Knowledge check

275. Are all allowed flows justified?
276. Can the design survive one instance failure?
277. Does the runbook separate connectivity, TLS and application failures?

Completion evidence: _____

Unit 46: Git and GitHub Interview Mastery

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Answer questions from fundamentals to architecture.
- Explain tradeoffs rather than memorized commands.
- Handle scenario interviews.

Why this matters in DevOps

Professional interviews evaluate reasoning, safety and collaboration as much as syntax.

Core question bank

- Explain working tree, index and repository.
- Merge versus rebase: when and why?
- Reset versus revert.
- How do you recover a deleted branch?
- How do you handle a committed secret?
- Design a branching strategy for frequent releases.
- How do protected branches and required checks work?
- How would you reduce a repository's clone time?
- How do you investigate which commit introduced a bug?
- How would you govern hundreds of repositories?

Answer framework

State the goal, clarify whether history is shared, identify risk, select the safest command or policy, describe validation and mention rollback.

Hands-on lab

278. Write answers to the ten core questions.
279. Record yourself explaining three scenarios.
280. Practice drawing commit graphs.

Knowledge check

281. Can you explain commands in terms of state transitions?
282. Do your answers mention safety and validation?
283. Can you compare multiple valid approaches?

Completion evidence: _____

Unit 47: Networking Interview Mastery

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Answer networking fundamentals.
- Troubleshoot whiteboard scenarios.
- Connect network behavior to cloud and containers.

Why this matters in DevOps

DevOps networking interviews test whether you can localize failures and communicate clearly.

Core question bank

- Explain what happens when a user opens an HTTPS URL.
- TCP versus UDP.
- How does DNS resolution work?
- What is CIDR and how do you calculate a subnet?
- Why can ping succeed while an application fails?
- Diagnose a TLS hostname error.
- Layer 4 versus Layer 7 load balancing.
- How does Kubernetes Service networking work conceptually?
- Public versus private cloud subnet.
- Design a zero-trust administrative path.

Troubleshooting answer framework

Clarify scope -> check recent changes -> test layers -> gather evidence -> mitigate reversibly -> validate
-> document root cause.

Hands-on lab

284. Answer the ten questions in writing.
285. Practice a five-minute URL walkthrough.
286. Troubleshoot three deliberately broken lab scenarios.

Knowledge check

287. Do you separate DNS, route, transport, TLS and HTTP?
288. Can you explain CIDR without guessing?
289. Do you mention security boundaries?

Completion evidence: _____

Unit 48: Final Practical Examination and Completion Review

Completion status: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Demonstrate integrated competence.
- Produce a professional evidence portfolio.
- Identify remaining weaknesses.

Why this matters in DevOps

Completion means you can perform, explain and troubleshoot the work without relying on memorized recipes.

Practical examination

- Create and govern a GitHub repository from scratch.
- Implement a feature and a hotfix using appropriate workflows.
- Resolve a merge conflict and recover a lost commit.
- Run CI checks and create a release.
- Design and calculate a multi-tier network.
- Diagnose DNS, TCP, TLS and HTTP failures.
- Create a packet-capture evidence note.
- Write an incident report and rollback procedure.

Passing standard

Complete all mandatory tasks, score at least 80% on theory review, and produce evidence that another learner can reproduce.

Hands-on lab

290. Complete the integrated examination in a disposable environment.
291. Create a portfolio folder with diagrams, commands, outputs and lessons learned.
292. Mark weak units for revision and repeat them.

Knowledge check

293. Can you reproduce every result?
294. Can you explain why each control exists?
295. Can another engineer follow your documentation?

Completion evidence: _____

Master Completion Tracker

Unit	Title	Status	Completion date
1	Version Control and DevOps Lifecycle	Not started	
2	Git Installation and Configuration	Not started	
3	Repository States	Not started	
4	Commit Design	Not started	
5	Git Internals	Not started	
6	Branches and HEAD	Not started	
7	Merging	Not started	
8	Rebase	Not started	
9	Undo and Recovery	Not started	
10	Tags and Releases	Not started	
11	Remote Repositories, Clone, Fetch, Pull and Push	Not started	
12	Stash, Clean and Worktree	Not started	
13	Cherry-pick, Bisect and Blame	Not started	
14	Git Ignore, Attributes, Line Endings and Binary Files	Not started	
15	Submodules, Subtrees and Monorepos	Not started	
16	Git Hooks, Aliases and Automation	Not started	
17	Git Security, Signing and Secret Prevention	Not started	
18	Repository Maintenance and Performance	Not started	
19	GitHub Foundations: Accounts, Repositories and Organizations	Not started	
20	Pull Requests and Professional Code Review	Not started	
21	Branch Protection, Rulesets and Governance	Not started	
22	GitHub Issues, Projects, Discussions and Documentation	Not started	
23	GitHub Actions Foundations	Not started	
24	GitHub Releases, Packages and Environments	Not started	
25	Enterprise GitHub Operations and API Automation	Not started	
26	Networking Fundamentals and the OSI/TCP-IP Models	Not started	
27	Ethernet, MAC Addresses, Switching and ARP	Not started	
28	IPv4 Addressing, Subnets and CIDR	Not started	
29	IPv6 Fundamentals	Not started	
30	Routing, Gateways, NAT and ICMP	Not started	
31	TCP and UDP	Not started	
32	DNS Architecture and Troubleshooting	Not started	
33	HTTP, HTTPS and API Networking	Not started	
34	TLS, Certificates and Public Key Infrastructure	Not started	
35	DHCP, Time, Proxies, VPNs and CDNs	Not started	
36	Linux Network Administration Commands	Not started	
37	Packet Capture with tcpdump and Wireshark	Not started	
38	Firewalls and Network Policy	Not started	
39	Load Balancers, Reverse Proxies and Service Discovery	Not started	
40	Docker and Kubernetes Networking Foundations	Not started	
41	Cloud Networking Foundations	Not started	

42	Network Security and Zero Trust	Not started	
43	Systematic Network Troubleshooting and Incident Response	Not started	
44	Capstone 1: Enterprise Git Collaboration	Not started	
45	Capstone 2: Network Architecture and Troubleshooting Lab	Not started	
46	Git and GitHub Interview Mastery	Not started	
47	Networking Interview Mastery	Not started	
48	Final Practical Examination and Completion Review	Not started	

Final Certification Checklist

- ☐ Every unit is marked completed.
- ☐ All mandatory labs were reproduced without copying line by line.
- ☐ Both capstone projects have diagrams, evidence and runbooks.
- ☐ Git and networking interview answers were practiced aloud.
- ☐ Final examination score is at least 80 percent.
- ☐ Weak areas were revised and retested.

Personal Completion Declaration

I confirm that I completed the practical work, can explain the core concepts, and understand that production changes require authorization, peer review, testing and rollback planning.

Name: _____ Date: _____

Signature: _____